

Finding Needles in the Haystack: Harnessing Syslogs for Data Center Management

Chen Liang
Duke University

Theophilus Benson
Duke University

Partha Kanuparth
Yahoo Research

Yihua He
Yahoo

ABSTRACT

Network device syslogs are ubiquitous and abundant in modern data centers with most large data centers producing millions of messages per day. Yet, the operational information reflected in syslogs and their implications on diagnosis or management tasks are poorly understood. Prevalent approaches to understanding syslogs focus on simple correlation and abnormality detection and are often limited to detection providing little insight towards diagnosis and resolution.

Towards improving data center operations, we propose and implement Log-Prophet, a system that applies a toolbox of statistical techniques and domain specific models to mine detailed diagnoses. Log-Prophet infers causal relationships between syslog lines and constructs succinct but valuable *problem graphs*, summarizing root causes and their locality, including cascading problems. We validate Log-Prophet using problem tickets and through operator interviews. To demonstrate the strength of Log-Prophet, we perform an initial longitudinal study of a large online service provider’s data center. Our study demonstrates that Log-Prophet significantly reduces the number of alerts while highlighting interesting operational issues.

1. INTRODUCTION

The performance of a content provider, and ultimately its revenue stream, is tied to the performance of its data center networks. Recognizing this trend, the networking community has proposed a number of architectures, frameworks and protocols to improve data center performance [8], and to diagnose reliability and performance problems [22, 2, 8]. Recent efforts on diagnosing data center problems have focused on using problem tickets [22] and host counters [33]. Surprisingly, network device syslogs, which have proved invaluable for diagnosing problems in ISP networks [28, 29, 15], have remained largely ignored in the data center space. Most notably, works focused on using syslogs to quantify and characterize physical failures [21, 19, 7, 20]. Instead of syslogs, other have used packet traces, SNMP, and ICMP to detect and diagnose problems; however, while highly powerful in detecting problems,

these techniques often provide minimal aide in problem resolution.

We argue that syslogs, unlike most other readily available management data (e.g. SNMP, pings, traceroutes), are imbued with sufficient information to not only diagnose problems but also to expedite problem resolution by highlighting critical properties of the problem. Despite their potential benefits, syslogs have been ignored because they lack structure (free form text), lack homogeneity (they differ across vendors and even for a vendor, they may differ across firmware versions), and lack global causality semantics that are necessary for diagnosis (information identifying causal relationships between a haystack of messages).

Current approaches to log analysis are inappropriate for syslogs for the following reasons. First, many require access to source code [31, 34]. Unfortunately, network devices run closed source, proprietary OSes.¹ Second, blackbox techniques [17, 13] employ simple correlation limiting their applicability and reducing accuracy.

In this work, we take the first step towards building a general log analysis framework that has two goals: (1) localize problems in the network (including problems that cascade between devices and protocol stack layers), and (2) diagnose root causes of problems (i.e., sequence of cascading events that caused the problem). Our system, Log-Prophet, aids operators by generating a structured representation of each problem: a causal graph of the cascading set of events (in unstructured syslogs). Log-Prophet mines causality in syslogs to generate this output, also called *Problem Graphs* (PGs). Our work builds on two insights. First, in data centers there is an abundance of logs that naturally lend themselves to statistical techniques. Second, network devices run networking protocols which dictate how problems propagate through the network: models of these protocols can be used to guide our statistical techniques, significantly improving problem troubleshooting.

Our goal is to extract from a stream of syslog messages, a set of accurate and precise PGs that enables

¹We note that even SDN capable devices come as closed source entities.

network operators to more readily answer, diagnosis, and resolve questions. In this paper we take a first step towards this goal, by presenting: a prototype of Log-Prophet; running the prototype on seven months worth of production syslog data from a large-scale data center network; and presenting a longitudinal study of the PGs discovered in the data center.

Our key contributions are:

- **Log-Prophet:** The design of a log analysis framework for data center networks that leverages domain knowledge and statistical technique to build problem graphs (PGs) (Section 4).
- **Prototype System:** A prototype implementation of Log-Prophet allowing us to validate our design and explore its benefits. (Section 5)
- **Study of PGs in a large Data center:** An initial analysis of the PGs generated by the data center of a large online service problem over a seven month period. We observe several interesting patterns about the PGs generated by devices in different layers of the data center (Section 6).

2. RELATED WORK

Log (and Syslog) Analysis. Previous studies of Syslog have focused on either understanding failures [21, 19, 7, 20, 28, 29], using syslogs as input to a machine learning algorithm for diagnosis [15], or developing techniques to extract structure from the syslogs [32, 23]. Our current study builds on existing approaches [32, 23] and leverages their techniques to structure the data enabling us to perform a wide range of analysis. Our analysis enriches existing work [21, 19, 7, 20, 28, 29] that focus on link and device failures by exploring the implications of causality between syslogs.

Orthogonal attempts to mine general system logs for diagnosis data often rely on access to source code [31, 34]. Unfortunately, most data center switches are closed source and thus we are unable to apply these techniques.

Statistical-based Problem Diagnosis Our approaches builds on correlation and causality, similarly other approaches have leveraged statical analysis to learn causal relationships or dependency graph from network data [12, 13, 14, 27, 11, 9, 30, 17]. We explore a different set of data sources with different features, which requires different models; for example, domain models to introduce structure and domain models to improve speed and accuracy of statistical techniques.

Distributed Tracing Distributed tracing [1, 3, 6, 25] has long been used in distributed systems as a way of building program graphs and using them to detect problems and their root causes. Unlike the RPC calls in distributed systems that include either accurate time, unique IDs that allow for tracing across devices, causality, or some combination, syslogs lack this information.

Specifically, there is no direct way to understand propagation of events across devices. To overcome this, we apply statistical techniques, guided by statistical techniques, to infer causality within and across devices.

3. DATA AND BACKGROUND

In this section, we present an overview of our datasets and the data center network’s operational patterns.

3.1 Data

We collected seven months worth of *syslog* data from one of Yahoo’s data centers. At a high-level, the data center network is a fat-tree topology consisting of three device layers (traversing bottom-up): Top of Rack (ToR) switches, two aggregation layers and a core layer. In addition, the data center includes middleboxes serving specialized functionality. We note that this general data center structure aligns with the set of data centers studied in the prior work [4, 10, 21, 19, 7, 20].

In Table 1, we describe the properties of these devices, focusing on their relative quantity, and summarizing the type and amount of data collected from each type of device. Unsurprisingly, we observe that the data center is dominated by edge or ToR switches. Interestingly, we observe a large amount of device heterogeneity: essentially there is heterogeneity at each layer of the data center with each layer consisting of devices from multiple vendors.

Network *syslog* messages (*syslogs*). *syslogs* are debugging messages about events and are generated by network devices to aid network operators in diagnosing network anomalies and in verifying the impact of maintenance operations. They capture a variety of network events ranging from failed user logins and problems with routing protocols to interface failures and CPU overheating. These messages are free-form text – essentially raw and unstructured.

The network devices are configured to push *syslogs* to one of several collection servers. We collect daily snapshots from these servers and aggregated these logs to develop our corpus. To analyze the data, we process syslogs to impose structure and enable diagnosis (we discuss the processing steps in Section 6).

Configuration. We collect nightly snapshots of device configurations, both static and device runtime outputs. We use a configuration management system that automatically captures these nightly snapshots. These configurations are instrumental in determining if observations within the *syslog* are due to transient changes or permanent changes. Moreover, the configuration allows us to reconstruct the topology and determine device and protocol adjacencies.

3.2 Domain-Specific Models

Log-Prophet builds on domain models to understand

Device Type	Devices	Description of devices	% of data center device types	# of Vendors	% of devices with config	% of devices with syslog
TOR	TOR_A, TOR_B, TOR_C	Top of Rack switches	93%	> 1	100%	25.5%
MB	MB_A, MB_B	Middleboxes	2%	> 1	100%	27.1%
AGG	AGG_A, AGG_B	Aggregation switches	4%	> 1	100%	16.1%
CORE	$CORE$	Core routers	0.07%	> 1	100%	100 %

Table 1: Overview of Dataset.

the cascading of events within and across devices. These models are one-time inputs and do not require operator feedback.

Protocol adjacencies. Network devices are configured to run a number of protocols such as BGP, OSPF, and MPLS. For each protocol, the device is configured to peer with other devices that are not physically adjacent. When a protocol-specific problem occurs, the event may propagate to a device’s logical peers. When an OSPF problem occurs on a device (e.g., OSPF process failure), the problem will only (initially) propagate to peers of this device (e.g. route timeouts on peers). Motivated by this observation, we build on our prior work [5] to parse network configurations and extend it to model these adjacencies. For each protocol in our data center, we create an *overlay graph* capturing these adjacencies.

Physical adjacencies. Similar to protocol adjacencies, problems occurring at the physical layer (layer 1) could only propagate to physically connected devices. We leverage network topology contained in configurations to add physical adjacencies.

Protocol delays. In addition to leveraging domain knowledge to model spatial propagation of events, Log-Prophet also uses domain knowledge to model temporal propagation of events across devices. Events propagate between protocol-adjacent devices in two ways; First, directly through the transfer of a message. For example, the establishment of a TCP connection for a peering session will generate syslog messages on both devices. Second, events can propagate indirectly due to the absence of an event before a timeout, which also generates a message. Examples of indirect propagation include an OSPF process failure on a device, which leads to the connection and routes on its peer to timeout several milliseconds later. In the first, messages are simultaneously generated on both devices. In the second, there is a lag between the two devices. To capture this, we parse configuration for timeout values and use these values to bound the time taken for protocol specific events to propagate across devices.

4. METHODOLOGY

The goal of Log-Prophet is to localize and find root causes of problems across the data center network. Some of these problems happen in parallel, especially at large-scale; and many of the problems cascade within

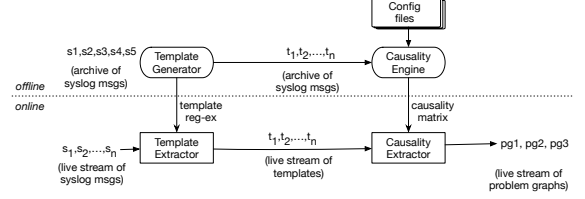


Figure 1: Diagram of Log-Prophet’s Workflow.

and between devices and across layers in protocol stack. A key to such a diagnosis system is to output a succinct but useful representation of a *problem* that aids operators. We define a problem as a directed graph, whose nodes are syslog messages (semantics) and whose edges are causal relationships between them (i.e., cascades) – we term this graph as a Problem Graph (PG). A PG describes everything that happened during a problem, starting from problems within a device and how they propagated to other device(s). As an example, a module failure (PHY) in a ToR device causes a link down problem (L2) and an OSPF problem (L3) within the device, which cause an interface (L2) issue in a connected AGG device.

In Figure 2, we present a high-level overview of Log-Prophet’s components and the interactions between these components. Log-Prophet consists of two classes of components: asynchronous low-volume preprocessing components which generate syslog templates and use that syslog structure to generate a causality matrix. The second, a set of online components ingest a live data center-wide stream of syslog messages, map them into a stream of templates, extract *casual rules* from the stream, and generate PGs from the extracted rules. The online components use relatively lightweight methods. Decoupling the offline and online components allows our techniques to scale to large data centers and enables us to parallelize execution in Log-Prophet.

4.1 Extracting Structure From syslogs

Several approaches have been proposed to impose structure on *syslogs* and extract “templates” from them. Templates are defined as short structured-format strings that succinctly describe the specific event embodied in a line of syslog as well as the entities involved in the event. In this study, we tried several approaches and settled on the one with the least error:

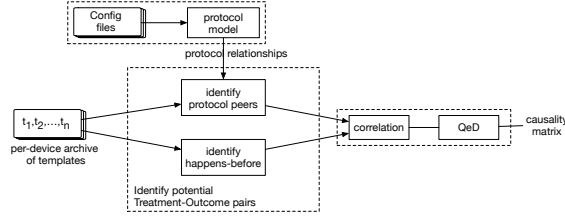


Figure 2: Causality Engine.

SysDigest [24].

4.2 Causality Engine

Log-Prophet’s causality engine, Figure 2, is an offline component of Log-Prophet that analyzes streams of templates to infer causality. The output of the causal engine is an $|n \times n|$ causality matrix M where M_{ij} is 1 if and only if template t_i has a causal impact on t_j .

In networks and distributed systems, the traditional approach for extracting causality from messages (or logs) is to implement logical timestamps in events and infer happens-before relationships. We could then use semantic information in the message (or logs) to track problems between devices. For example, Sherlock uses IP addresses in packets to determine the flow of causality where as Mystery Machine learns causality between RPCs based on happens-before relationships. These methods either require precise timing information from synchronized clocks, and/or significant instrumentation to add information such as logical clocks to device events. Since we cannot modify (closed source) network devices to add instrumentation such as logical clocks or record fine-grained timestamps, we cannot use traditional methods; further, happens-before relationships cannot be inferred from syslogs in a large-scale network (the needle in a haystack problem).

To address these challenges, our causality engine leverages the fact that data centers generate millions of events per day and are thus amenable to statistical techniques to find causality (a stronger definition than happens-before). We apply Quasi-Experimental Design, QED, a popular technique used by social scientists to find causal relationships in data sets [26, 16]. QED is often used in the absence of traditional data sources and controlled experiments such as A/B testing. QED establishes a causal relationship between pairs of templates (from same or different devices), a treatment variable (cause), t_i , and an outcome variable (effect), t_j , by using history; and accounts for confounding factors. For example, using QED we are able to affirm a causal relationship between a link failure (treatment) and a routing protocol outage (outcome), conditioning for confounding factors such as device config changes, power, firmware, and operating system. Using QED,

we are able to build a statistically significant causality matrix M .

4.3 QED

At a high level, QED detects causality between an independent variable X “treatment” and a dependent variable Y “outcome” by identifying a set of “untreated” variables $Z = z_1 \dots z_n$ that are identical to X , after accounting for confounding factors [26]. The key to accurately affirming causality, lies in precisely identifying all confounding factors and asserting that X is identical to elements of Z – this assertion allows us to affirm that any difference in outcomes between paired experiments on $z_i \subset Z$ and X is entirely due to X .

Thus to apply QED, the causality engine must take the following steps: (1) identify dependent variables in our data set, (2) control for a pre-defined set of confounding factors, (3) identify “untreated” variables for each potential causal relationship, and (4) perform a statistical test of relevance.

Identifying dependent variables. To identify dependent variables, we leverage domain knowledge discussed in Section 3.2 to identify templates across devices that may be dependent. Specifically, two templates across devices may be dependent if they have spatial and temporal adjacencies. For templates within a device, we also leverage happens-before relationships, because syslog stream from a device is strictly ordered. Once we identify the sets of potentially dependent variables we apply nonparametric correlation to ensure statistical dependence: we correlate pairs of templates and identify two templates t_i and t_j as dependent if the correlation value is significant (over α).

Identifying confounding factors. The confounding factors pair templates t_i and t_j conditioned on external factors that impact t_i and t_j . These confounding factors include device vendor, OS, location in data center topology, and configuration changes. Thus for each template t_i , we identify a confounding set $K = t_j \dots t_m$ such that each element $t_j \in K$ is correlated with t_i ($C_{ij} > \alpha$) and they share identical confounding factors.

Identifying untreated variables. Given the confounding set K an “untreated” template, t_u , is a randomly selected member of K .

Statistical test of relevance. Next, we compare the untreated template, t_u , and with the treated template, t_i . Using historical data, Log-Prophet performs a hypothesis test for the null hypothesis (H_0) that t_t has no impact on t_o and calculate the p -value. Based on the test, we affirm that t_t (treatment) has a causal impact on t_o (outcome).

4.4 Constructing problem graphs (PGs)

Given a causality matrix, M , Log-Prophet can pro-

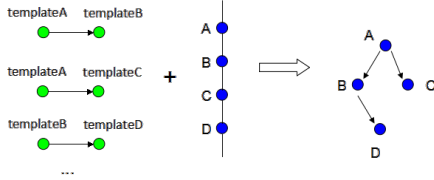


Figure 3: Causally related template pairs and a stream of templates are used to build a PG.

cess a stream of templates and identify pair of templates t_j and t_i where t_j causally impacts t_i (illustrated in Figure 3). We summarize the stream as a set of PGs, encoding a set of causally related templates.

More formally, a PG is a graph $G = (V, E)$ with one vertex per syslog template and directed edges between syslog templates representing causality. A directed edge in a PG represents a causal relationships between two templates (or syslog lines) with the direction indicating causality. Figure 4 shows two examples of problems from our data center (colors encode layer in protocol stack). The left PG shows an example of a multi-layer problem that spans AGG and ToR layers and multiple layers in the protocol stack. It encodes templates that capture a module failure template that causes a link down template, which further triggers a spanning tree protocol status change template, and at the time causing a interface status change template on a peering device. An edge from a node to itself implies that multiple, often duplicate, syslog lines are often generated by a specific event: the edge between “LineProtocol” and itself indicates that when LineProtocol fails, devices generate multiple syslog lines. The right PG shows an example of a problem within ToR devices that is an Ethernet (L2) flapping issue.

To construct PGs (illustrated in Figure 3), Log-Prophet processes a stream of syslog templates online, and divides the stream into time windows of templates of size δ . For each window, it creates vertices for each template and adds them into graph P . It adds a direct edge between $i \in P$ and $j \in P$ if M_{ij} is 1. Finally, Log-Prophet scans P for duplicate vertices x and y such that $template(x) == template(y)$. It merges the nodes and preserves the directed edges. If an edge exists between y and x , then it creates a self loop.

Broadly, when used for problem determination, the key aspect of the PG are the edges and the roots of the graph. The roots allows operators to determine (trace back) potential root cause(s) and the edges enable operators to understand the escalation/propagation of the problem.

5. PROTOTYPE AND VALIDATION

We developed an initial prototype of Log-Prophet in

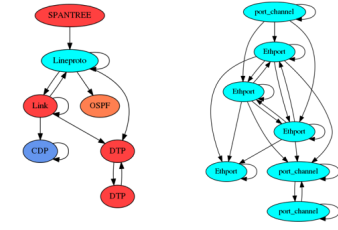


Figure 4: Example PGs from our data center.

Python in approximately 4000 LoCs. Our prototype builds on the statistical libraries provided by scikit-learn [18] for statistical analysis. We include models for the following protocols: BGP, VLAN, and OSPF.

Validation: To validate Log-Prophet, we examined the problem ticket database for the data center and interviewed operators of our data centers. For our operator interviews, we analyzed a subset of the data: 6 weeks worth of data. We presented the raw syslogs along with the discovered PGs, and discussed the information contained within the PGs as well potential problem tickets generated for these graphs. Our initial results are promising – the operators agreed that the PGs succinctly captured events within a device. For events across devices, we were limited by the set of protocols we have currently modeled. We plan to extend our prototype with protocol models. We are also working on a systematic validation of Log-Prophet output, both using live data (in collaboration with network operations), and an emulation testbed.

6. INITIAL CHARACTERIZATION OF PRODUCTION PGs

In this section, we present a longitudinal analysis of syslogs and resulting PGs for a large data center consisting of over five thousand devices. In analyzing these PGs, we aim to understand: (1) the data center’s operational dynamics from the perspective of the network devices; (2) the persistence of operations issues (as captured by PGs); and (3) the locality of operational issues to specific types of devices or layers in the networking stack.

6.1 Classification of PGs

In applying Log-Prophet on seven months of data, we discovered 4264 PGs which could be easily clustered into 156 classes: through manual inspection we were able to classify these graphs into 22 high level types of PGs. The manual inspection allowed us to group together graphs consisting of templates from different vendors – the clustering couldn’t not do this as their graphs were included radically different templates.

We observe that applying Log-Prophet to our data

center allows us to reduce hundreds of millions of lines into 13 distinct PGs with each PG. We analyzed a time series of the number of syslogs generated and compare them against the number of instances of PGs extracted (not shown due to space constraints). From this we observe that while there are only 13 unique PGs in our data center: there are in fact thousands of events occurring on a daily basis. From an operational standpoint, this is a more tractable number than the hundreds of millions of syslogs. Furthermore, while there are thousands of instances PGs they only map to thirteen unique PGs thus problem determination and resolution will involve little overhead.

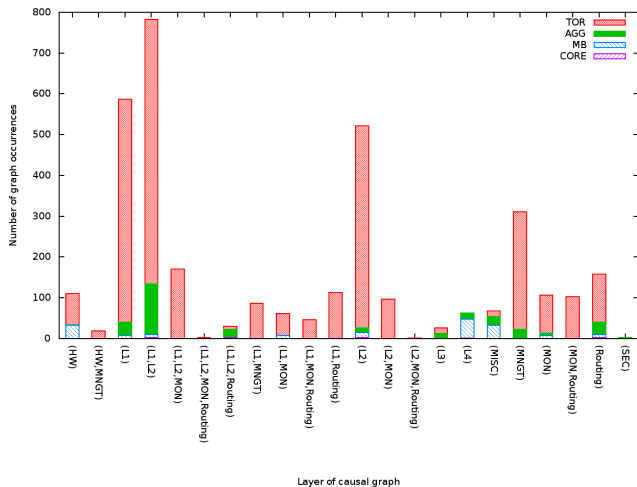


Figure 5: Number of PGs for each layer.

6.2 Footprint of PGs

Next in Figure 5, we analyze the breakdown of PGs across the different layers in the data center. There are several interesting observations about the distribution of graphs across data centers layers.

ToR layer dominates: Over 93% of the PGs occur at the ToR devices. This pattern is a direct manifestation of two trends: first, over 90% of the devices in the data center are ToR devices (See Table 1); second, ToR devices are often cheap commodity devices with lower quality parts and are thus prone to more failures [19, 7].

L3-L4 PGs dominate MBs: Unlike the ToR and the AGG layers, the middlebox layer are dominated by a combination of L4 PGs (virtual IP migration), routing PGs, and HW PGs. In the the different types of MB only generate 2-3 graphs: for example, the load balancer generates PGs related to: virtual IP migration (or failures); memory issues; and interface failures creating IP unreachability.

PGs across devices: A number of interesting PGs display problems that cross devices boundaries and even

tier boundaries. For example, we find that graphs related to interface failures in a ToR often percolate up to affect the OSPF routing protocol in adjacent AGG devices (illustrated in Figure 4).

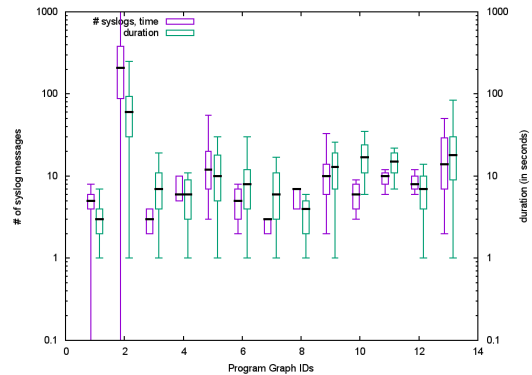


Figure 6: Number of PGs for each layer.

Lastly in Figure 6, we analyze the size of a representative subset of the PGs in terms of duration (in seconds) and length (number of messages covered). Due to space we limit the figure to 14 representative PGs. Naturally, there exists a diversity in the sizes of the different PG. More interestingly, we observe that counter intuitively, there is no direct correlation between duration and syslog lines capture: this indicates that simply looking at number of syslogs is insufficient.

Takeaways: Although our results are preliminary, they provide us with an overview of the problems with modern data centers. We are currently working with operators to explore methods for developing tools to automatically suppress superfluous PGs; rank PGs based on importance and scope; and to use PGs to suggest potential resolutions.

7. CONCLUSION

There exists a dichotomy within the community. On one hand, a large number of studies have characterized data center failures and performance issues using a specific set of syslog messages (i.e. interface failures). On the other hand, extensive work within carrier and ISP networks have displayed the benefits of using a larger set of syslog messages. In this work, we take the first step towards reconciling this dichotomy by developing Log-Prophet, a system for analyzing syslog messages and extracting problem graphs that summarize causality within the data center. We have validated Log-Prophet with network operators and applied to an initial data center. Our initial analysis are promising and they show that there is a strong benefit in employing frameworks such as Log-Prophet.

8. REFERENCES

- [1] M. K. Aguilera, J. C. Mogul, J. L. Wiener, P. Reynolds, and A. Muthitacharoen. Performance debugging for distributed systems of black boxes. In *ACM SIGOPS Operating Systems Review*, volume 37, pages 74–89. ACM, 2003.
- [2] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan. Data center tcp (dctcp). In *Proceedings of SIGCOMM*, 2010.
- [3] M. Attariyan, M. Chow, and J. Flinn. X-ray: Automating root-cause diagnosis of performance anomalies in production software. In *OSDI*, pages 307–320, 2012.
- [4] T. Benson, A. Akella, and D. A. Maltz. Network traffic characteristics of data centers in the wild. In *Proceedings of IMC*, 2010.
- [5] T. Benson, A. Akella, and A. Shaikh. Demystifying configuration challenges and trade-offs in network-based isp services. In *Proceedings of the ACM SIGCOMM 2011 Conference*, SIGCOMM ’11, pages 302–313, New York, NY, USA, 2011. ACM.
- [6] R. Fonseca, G. Porter, R. H. Katz, S. Shenker, and I. Stoica. X-trace: A pervasive network tracing framework. In *Proceedings of the 4th USENIX conference on Networked systems design & implementation*, pages 20–20. USENIX Association, 2007.
- [7] P. Gill, N. Jain, and N. Nagappan. Understanding network failures in data centers: Measurement, analysis, and implications. In *Proceedings of SIGCOMM*, 2011.
- [8] A. Greenberg, J. R. Hamilton, N. Jain, S. Kandula, C. Kim, P. Lahiri, D. A. Maltz, P. Patel, and S. Sengupta. V12: A scalable and flexible data center network. In *Proceedings of SIGCOMM*, 2009.
- [9] S. Kandula, R. Chandra, and D. Katabi. What’s going on?: learning communication rules in edge networks. In *ACM SIGCOMM Computer Communication Review*, volume 38, pages 87–98. ACM, 2008.
- [10] S. Kandula, S. Sengupta, A. Greenberg, P. Patel, and R. Chaiken. The nature of data center traffic: Measurements & analysis. In *Proceedings of IMC*, 2009.
- [11] R. R. Kompella, J. Yates, A. Greenberg, and A. C. Snoeren. Ip fault localization via risk modeling. In *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2*, pages 57–70. USENIX Association, 2005.
- [12] S. S. Krishnan and R. K. Sitaraman. Video stream quality impacts viewer behavior: inferring causality using quasi-experimental designs. *Networking, IEEE/ACM Transactions on*, 21(6):2001–2014, 2013.
- [13] A. Lakhina, M. Crovella, and C. Diot. Diagnosing network-wide traffic anomalies. In *ACM SIGCOMM Computer Communication Review*, volume 34, pages 219–230. ACM, 2004.
- [14] A. Mahimkar, J. Yates, Y. Zhang, A. Shaikh, J. Wang, Z. Ge, and C. T. Ee. Troubleshooting chronic conditions in large ip networks. In *Proceedings of the 2008 ACM CoNEXT Conference*, page 2. ACM, 2008.
- [15] A. A. Mahimkar, H. H. Song, Z. Ge, A. Shaikh, J. Wang, J. Yates, Y. Zhang, and J. Emmons. Detecting the performance impact of upgrades in large operational networks. In *Proceedings of SIGCOMM*, 2010.
- [16] H. Oktay, B. J. Taylor, and D. D. Jensen. Causal discovery in social media using quasi-experimental designs. In *Proceedings of the First Workshop on Social Media Analytics*, pages 1–9. ACM, 2010.
- [17] A. J. Oliner, A. V. Kulkarni, and A. Aiken. Using correlated surprise to infer shared influence. In *DSN*, pages 191–200. IEEE Computer Society, 2010.
- [18] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [19] R. Potharaju and N. Jain. Demystifying the dark side of the middle: A field study of middlebox failures in datacenters. In *Proceedings of IMC*, 2013.
- [20] R. Potharaju and N. Jain. An empirical analysis of intra- and inter-datacenter network failures for geo-distributed services. In *Proceedings of SIGMETRICS*, 2013.
- [21] R. Potharaju and N. Jain. When the network crumbles: An empirical study of cloud network failures and their impact on services. In *Proceedings of SoCC*, 2013.
- [22] R. Potharaju, N. Jain, and C. Nita-Rotaru. Juggling the jigsaw: Towards automated problem inference from network trouble tickets. In *Proceedings of nsdi*, 2013.
- [23] T. Qiu, Z. Ge, D. Pei, J. Wang, and J. Xu. What happened in my network: Mining network events from router syslogs. In *Proceedings of IMC*, 2010.
- [24] T. Qiu, Z. Ge, D. Pei, J. Wang, and J. Xu. What happened in my network: Mining network events from router syslogs. In *Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement*, IMC ’10, pages 472–484, New York, NY, USA, 2010. ACM.
- [25] P. Reynolds, C. E. Killian, J. L. Wiener, J. C. Mogul, M. A. Shah, and A. Vahdat. Pip: Detecting the unexpected in distributed systems. In *NSDI*, volume 6, pages 115–128, 2006.
- [26] W. R. Shadish, T. D. Cook, and D. T. Campbell. *Experimental and quasi-experimental designs for generalized causal inference*. Wadsworth Cengage learning, 2002.
- [27] M. Tariq, A. Zeitoun, V. Valancius, N. Feamster, and M. Ammar. Answering what-if deployment and configuration questions with wise. In *ACM SIGCOMM Computer Communication Review*, volume 38, pages 99–110. ACM, 2008.
- [28] D. Turner, K. Levchenko, S. Savage, and A. C. Snoeren. A comparison of syslog and is-is for network failure analysis. In *Proceedings of IMC*, 2013.
- [29] D. Turner, K. Levchenko, A. C. Snoeren, and S. Savage. California fault lines: Understanding the causes and impact of network failures. In *Proceedings of SIGCOMM*, 2010.
- [30] T. Wang, M. Srivatsa, D. Agrawal, and L. Liu. Learning, indexing, and diagnosing network faults. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 857–866. ACM, 2009.
- [31] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan. Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pages 117–132. ACM, 2009.
- [32] K. Yamanishi and Y. Maruyama. Dynamic syslog mining for network failure monitoring. In *Proceedings of KDD*, 2005.
- [33] M. Yu, A. Greenberg, D. Maltz, J. Rexford, L. Yuan, S. Kandula, and C. Kim. Profiling network performance for multi-tier data center applications. In *Proceedings of NSDI*, 2011.
- [34] X. Zhao, Y. Zhang, D. Lion, M. F. Ullah, Y. Luo, D. Yuan, and M. Stumm. lprof: A non-intrusive request flow profiler for distributed systems. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 629–644, Broomfield, CO, Oct. 2014. USENIX Association.